HW #4.3: Reproducible Data Analysis

Ata Dural

2026-04-13

Busiest Airports Analysis

This section compares six of the busiest passenger airports in the world from 2020 to 2025. I use both a table and a line graph because they help in different ways. The table is better for checking exact passenger counts, while the graph is better for seeing overall patterns across time.

Table 1: Passenger traffic for six major airports from 2020 to 2025.

Airport	IATA	2020	2021	2022	2023	2024	2025
Hartsfield-Jackson Atlanta International Airport	ATL	42,918,685	75,704,760	93,699,630	104,653,451	108,067,766	106,302,208
Frankfurt Airport	FRA	18,768,601	24,812,849	48,918,482	59,355,389	61,564,957	63,189,666
Beijing Daxing International Airport	PKX		25,051,012			49,441,029	53,618,949
Istanbul Airport	IST	23,330,411	36,988,563	64,289,107	76,027,321	80,073,252	84,457,421
King Abdulaziz International Airport	JED			31,648,324	42,910,407	49,072,000	53,400,000
Tokyo Haneda Airport	HND	31,055,210	25,876,429	50,290,705	78,719,302	85,900,167	91,430,000

Table 1 and Figure 1 work together to show both the exact values and the broader pattern. The table makes it easy to compare specific passenger counts from year to year, while the line graph shows how traffic changed over time for each airport. Looking at both together gives a clearer picture than using only one of them.

The strongest pattern is that Hartsfield Jackson Atlanta International Airport stays above the others across the whole period. In the graph, its line remains the highest in every year, and the table confirms that it has the largest passenger totals throughout the data. This makes Atlanta the clearest example of the busiest airport in the group.

Tokyo Haneda Airport and Istanbul Airport show some of the strongest growth across the years. Their lines rise sharply after the earlier years, which makes the recovery easy to notice in the graph. The table supports that same point by showing much larger passenger counts in the later years than in the earlier ones. Frankfurt Airport also increases, but its line rises in a steadier and less dramatic way, which suggests more gradual growth.

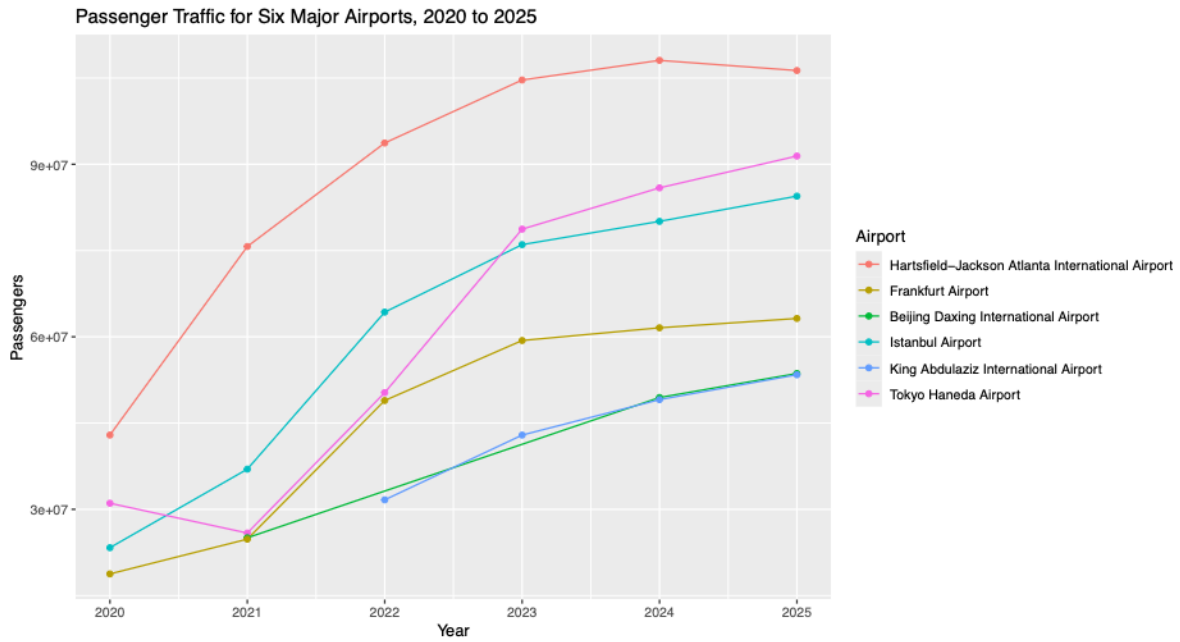
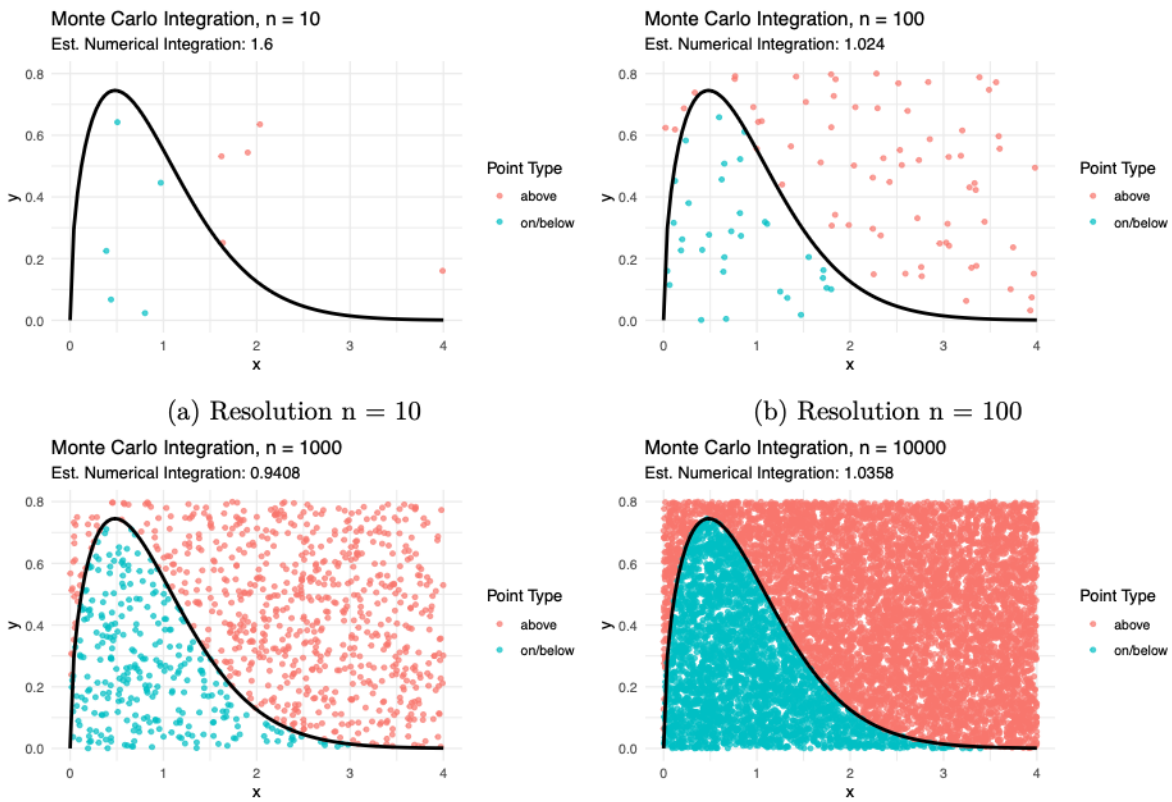


Figure 1: Passenger traffic trends for six major airports from 2020 to 2025.

The two visuals also help explain gaps in the data. Beijing Daxing International Airport has missing values in 2022 and 2023, and King Abdulaziz International Airport begins in 2022 rather than 2020. Those breaks are important because they affect how the lines appear and how the table should be read. Even with those missing years, both airports still show growth in the years where data are available.

Overall, the table is most useful for checking exact numbers, while the graph is better for seeing recovery, growth, and separation between airports. Together they show that passenger traffic generally increased after the earlier years, but the pace of that increase was not the same for every airport. Atlanta remained the busiest, Tokyo and Istanbul grew strongly, Frankfurt rose more steadily, and the missing years for Beijing and the later start for King Abdulaziz are important parts of the story.

Monte Carlo Numerical Integration



(a) Resolution n = 10

(b) Resolution n = 100

(c) Resolution n = 1000

(d) Resolution n = 10000

Figure 2: Monte Carlo numerical integration for the Weibull probability density function at four different resolutions.

Monte Carlo numerical integration estimates the area under a curve by using random points inside a rectangular region. For this problem, the function is the Weibull probability density function, and the rectangle uses x values from 0 to 4 and y values from 0 to 0.8. That means the rectangular window has area ($4 \times 0.8 = 3.2$). Random points are generated inside that window, and each point is checked to see whether it falls on or below the curve or above it. The proportion of points on or below the curve is then multiplied by 3.2 to estimate the integral.

Figure 2 shows the same Monte Carlo process at four different resolutions: 10, 100, 1000, and 10000 points. The smaller panels are more irregular because there are fewer sampled points, so the estimate can move around more from one run to another. As the number of points increases, the cloud of points fills the rectangle more evenly and the estimated value becomes more stable. That makes it easier to see the shape of the region under the Weibull curve and

how Monte Carlo simulation improves with more sampled points.

The exact answer to this numerical integration is the area under the Weibull density curve from 0 to 4. In R, that exact value can be written as `pweibull(4, shape = 1.5, scale = 1) - pweibull(0, shape = 1.5, scale = 1)`, which is about 0.9997. I think the exact answer is therefore very close to 1. The strongest support for that conclusion comes from the higher resolution panels in Figure 2, especially the 1000 point and 10000 point plots, because those estimates should settle closer to the true value and vary less than the lower resolution plots. That pattern helps show both how the method works and why the estimate near 1 is the most reasonable conclusion.

Planning and Prompting GenAI Tools

The goal is to take the calcium data and turn it into a tidy format so that I can clearly compare how calcium levels change over time for the two groups. One group did not receive any treatment, while the other group followed a diet and exercise program. I want a graph that makes it easy to see how these two groups change from the initial measurement to Year 3.

To do this, I need to use the calcium CSV file as it is. The data are not tidy because each group is stored in separate columns instead of having one column for treatment group and one column for time. I need to fix that before making any graph.

My plan is to first read in the file and identify which columns belong to the null group and which belong to the treatment group. Then I will rename the columns so the time points are clear. After that, I will reshape the data from wide format into long format so that each row represents one person at one time point. I will create a new column for treatment group and another for time.

Once the data are tidy, I will make a graph that shows calcium levels across time for each group. A line graph will work well because it shows change over time. I will label the axes clearly and include a legend so it is easy to compare the two groups. Finally, I will check that the graph actually shows whether one group maintains higher calcium levels over time.

Plan Based Prompt

I used ChatGPT and gave it a prompt based on my plan.

The prompt I used was:

"Using the calcium dataset, reshape the data into a tidy format so that each row represents a single observation with columns for treatment group, time, and calcium value. Then create a line graph that shows how calcium levels change over time for both the treatment group and the control group. Make sure the graph clearly compares the two groups across all time points."

The response I received was:

The tool explained how to first reshape the data from wide format into long format so that each row represents one observation. It suggested creating a column for time and another for treatment group so that the data can be properly organized.

It then generated code to pivot the dataset into a tidy structure. After that, it created a line graph showing calcium levels over time, with separate lines for the treatment group and the control group. The graph included clear axis labels and a legend to distinguish between the two groups.

The result made it easy to compare how calcium levels changed from the starting point to Year 3 for both groups.

Generic Prompt Response

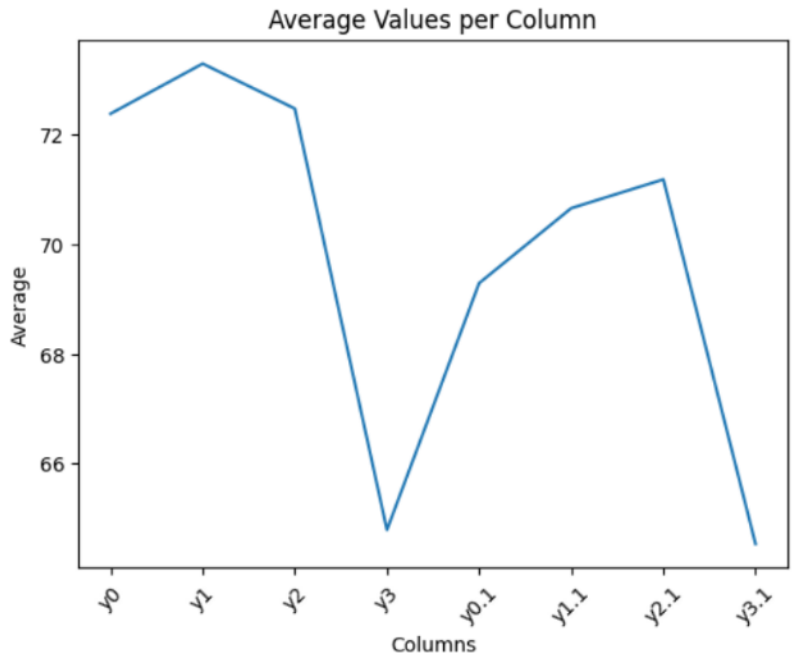


Figure 3: Average values for each column from the calcium dataset using a generic prompt.

Comparison

The generic response produced a plot showing the average value of each column in the dataset. Each point represents the mean of a column such as y0, y1, and the later columns. Most of the values are around the low 70s, while a few drop closer to the mid 60s. This gives a quick overview of the data but does not show how values change over time. The plot treats each column as separate, so it does not reflect how calcium levels change across time or between the two groups.

The plan-based response and the generic response are very different in how useful they are. The plan-based response focuses on the structure of the dataset and the goal of comparing calcium levels over time between two groups. It reshapes the data so that time and treatment group are clearly represented, which makes the final graph meaningful.

The generic response does not do this. It only calculates averages for each column and treats them as separate values. Because of that, it does not show how calcium levels change over time or how the two groups compare.

Another key difference is that the plan-based response uses the context of the study, while the generic response ignores it. The plan-based output directly answers the main question, while the generic one only gives a surface level summary.

This shows that a clear and specific prompt leads to a much better result. When the instructions match the structure of the data, the output becomes more useful and easier to understand.

Reflection

Working on this assignment helped me understand how important it is to give clear and structured instructions when using tools like ChatGPT. At first, I thought a simple prompt would be enough, but the generic response showed me that without context, the output can miss the main goal of the problem.

One thing I learned is how the structure of the data affects the results. Before this, I did not fully understand why reshaping data into long format matters, but after seeing both outputs, it became clear that organizing the data properly makes a big difference in how useful the graph is.

I also realized that planning before writing a prompt leads to much better results. When I followed my plan, the output directly answered the question and made it easy to compare the two groups over time. This showed me that taking time to think through the steps first is important, especially when working with data.

Overall, this assignment helped me improve how I approach both data analysis and using AI tools. I learned that better inputs lead to better outputs, and that being clear and specific is key.

GenAI Usage Appendix

Tool Used: ChatGPT (OpenAI, GPT-5.3)
Date: April 2026

Usage 1: Monte Carlo explanation

Prompt:
Explain how to simulate random points and estimate area under a curve using Monte Carlo in R. Keep it simple and walk me through.

Response:
The model explained how to generate random x and y values using runif, compare them to a curve, and estimate the area based on the proportion of points below the curve. It described looping through values and counting how many points fall under the curve.

Usage 2: Plot interpretation

Prompt:
Help me describe what this Monte Carlo simulation plot eli5

Response:
The model explained that the plot shows random points and highlights which ones fall below the curve. It also explained that the ratio of those points can be used to estimate the area under the curve.

Usage 3: Reflection writing

Prompt:
Help me write a reflection comparing a plan based response and a generic response

Response:
The model helped structure a reflection explaining differences between the two approaches, focusing on clarity, structure, and usefulness of results.

Usage 4: Planning and Prompting section

Prompt:
Using the calcium dataset, reshape the data into a tidy format so that each row represents a single observation with columns for treatment group, time, and calcium value. Then create a line graph that shows how calcium levels change over time for both the treatment group and the control group. Make sure the graph clearly compares the two groups across all time points.

Response:
The tool explained how to first reshape the data from wide format into long format so that each row represents one observation. It suggested creating a column for time and another for treatment group so that the data can be properly organized.

It then generated code to pivot the dataset into a tidy structure. After that, it created a line graph showing calcium levels over time, with separate lines for the treatment group and the control group. The graph included clear axis labels and a legend to distinguish between the two groups.

The result made it easy to compare how calcium levels changed from the starting point to Year 3 for both groups.

Code Appendix

Busiest Airports Analysis

This section gathers, cleans, and organizes airport traffic data.

```
airport_order <- c(
  "Hartsfield-Jackson Atlanta International Airport",
  "Frankfurt Airport",
  "Beijing Daxing International Airport",
  "Istanbul Airport",
  "King Abdulaziz International Airport",
  "Tokyo Haneda Airport"
)

code_order <- c("ATL", "FRA", "PKX", "IST", "JED", "HND")

year_order <- c(2020, 2021, 2022, 2023, 2024, 2025)
```

```r
add_commas <- function(x) {
  if (is.na(x)) {
    answer <- ""
  } else {
    answer <- format(x, big.mark = ",", scientific = FALSE, trim = TRUE)
  }

  answer
}

airport_col <- c()
iata_col <- c()
y2020 <- c()
y2021 <- c()
y2022 <- c()
y2023 <- c()
y2024 <- c()
y2025 <- c()

i <- 1
while (i <= length(airport_order)) {
  current_airport <- airport_order[i]
  current_code <- code_order[i]

  airport_col <- c(airport_col, current_airport)
  iata_col <- c(iata_col, current_code)

  one_2020 <- NA
  one_2021 <- NA
  one_2022 <- NA
  one_2023 <- NA
  one_2024 <- NA
  one_2025 <- NA

  j <- 1
  while (j <= nrow(busiest_airports_tidy)) {
    same_airport <- FALSE
    same_year <- FALSE

    if (busiest_airports_tidy$airport[j] == current_airport) {
      same_airport <- TRUE
    } else {
```

```r
    same_airport <- FALSE
  }

  if (same_airport == TRUE) {
    if (busiest_airports_tidy$year[j] == 2020) {
      same_year <- TRUE
      one_2020 <- busiest_airports_tidy$passengers[j]
    } else {
      if (busiest_airports_tidy$year[j] == 2021) {
        same_year <- TRUE
        one_2021 <- busiest_airports_tidy$passengers[j]
      } else {
        if (busiest_airports_tidy$year[j] == 2022) {
          same_year <- TRUE
          one_2022 <- busiest_airports_tidy$passengers[j]
        } else {
          if (busiest_airports_tidy$year[j] == 2023) {
            same_year <- TRUE
            one_2023 <- busiest_airports_tidy$passengers[j]
          } else {
            if (busiest_airports_tidy$year[j] == 2024) {
              same_year <- TRUE
              one_2024 <- busiest_airports_tidy$passengers[j]
            } else {
              if (busiest_airports_tidy$year[j] == 2025) {
                same_year <- TRUE
                one_2025 <- busiest_airports_tidy$passengers[j]
              } else {
                same_year <- FALSE
              }
            }
          }
        }
      }
    }
  } else {
    same_year <- FALSE
  }

  j <- j + 1
}
```

```r
  y2020 <- c(y2020, add_commas(one_2020))
  y2021 <- c(y2021, add_commas(one_2021))
  y2022 <- c(y2022, add_commas(one_2022))
  y2023 <- c(y2023, add_commas(one_2023))
  y2024 <- c(y2024, add_commas(one_2024))
  y2025 <- c(y2025, add_commas(one_2025))

  i <- i + 1
}

airport_table <- data.frame(
  Airport = airport_col,
  IATA = iata_col,
  `2020` = y2020,
  `2021` = y2021,
  `2022` = y2022,
  `2023` = y2023,
  `2024` = y2024,
  `2025` = y2025,
  stringsAsFactors = FALSE,
  check.names = FALSE
)

library(kableExtra)

knitr::kable(
  airport_table,
  align = "lcccccc",
  booktabs = TRUE
) |>
  kableExtra::kable_styling(
    latex_options = c("hold_position", "scale_down"),
    font_size = 6
  )
```

```r
library(ggplot2)

plot_year <- c()
plot_airport <- c()
plot_passengers <- c()

i <- 1
```

```r
while (i <= nrow(busiest_airports_tidy)) {
  if (!is.na(busiest_airports_tidy$passengers[i])) {
    plot_year <- c(plot_year, busiest_airports_tidy$year[i])
    plot_airport <- c(plot_airport, busiest_airports_tidy$airport[i])
    plot_passengers <- c(plot_passengers, busiest_airports_tidy$passengers[i])
  }

  i <- i + 1
}

plot_data <- data.frame(
  year = plot_year,
  airport = plot_airport,
  passengers = plot_passengers,
  stringsAsFactors = FALSE
)

plot_data$airport <- factor(
  plot_data$airport,
  levels = c(
    "Hartsfield-Jackson Atlanta International Airport",
    "Frankfurt Airport",
    "Beijing Daxing International Airport",
    "Istanbul Airport",
    "King Abdulaziz International Airport",
    "Tokyo Haneda Airport"
  )
)

ggplot(plot_data, aes(x = year, y = passengers, color = airport, group = airport)) +
  geom_line() +
  geom_point() +
  labs(
    title = "Passenger Traffic for Six Major Airports, 2020 to 2025",
    x = "Year",
    y = "Passengers",
    color = "Airport"
  )
```

Monte Carlo Numerical Integration

This section generates random points and estimates area under the curve.

```
library(ggplot2)

run_mc_sim <- function(n_points, x_min, x_max, y_min, y_max) {
  x_values <- c()
  y_values <- c()

  i <- 1
  while (i <= n_points) {
    x_values <- c(x_values, runif(1, x_min, x_max))
    y_values <- c(y_values, runif(1, y_min, y_max))
    i <- i + 1
  }

  sim_data <- data.frame(x = x_values, y = y_values)

  sim_data
}

make_mc_plot <- function(n_points) {
  mc_data <- run_mc_sim(n_points, 0, 4, 0, 0.8)

  flag_values <- c()
  below_count <- 0

  i <- 1
  while (i <= nrow(mc_data)) {
    x_now <- mc_data$x[i]
    y_now <- mc_data$y[i]
    curve_y <- dweibull(x_now, shape = 1.5, scale = 1)

    if (y_now <= curve_y) {
      flag_values <- c(flag_values, "on/below")
      below_count <- below_count + 1
    } else {
      flag_values <- c(flag_values, "above")
    }
```

```r
    i <- i + 1
  }

  mc_data$flag <- flag_values

  box_area <- 4 * 0.8
  prop_below <- below_count / nrow(mc_data)
  est_int <- box_area * prop_below

  label_text <- paste("Est. Numerical Integration:", round(est_int, 4))

  ggplot(mc_data, aes(x = x, y = y, color = flag)) +
    geom_point(alpha = 0.7, size = 1.2) +
    stat_function(
      fun = dweibull,
      args = list(shape = 1.5, scale = 1),
      xlim = c(0, 4),
      linewidth = 1.1,
      inherit.aes = FALSE,
      color = "black"
    ) +
    labs(
      title = paste("Monte Carlo Integration, n =", n_points),
      subtitle = label_text,
      x = "x",
      y = "y",
      color = "Point Type"
    ) +
    coord_cartesian(xlim = c(0, 4), ylim = c(0, 0.8)) +
    theme_minimal()
}
```